K

KAILEN AI

WINDOWS DESKTOP SOFTWARE DOSSIER



Complete Software Documentation

Product, usage, architecture, security, configuration, provider integrations, theme studio, and development guide.

SOURCE BASELINE • v2.8.0 • 23 OCTOBER 2026

Prepared from the supplied Kailen AI v2.8 source archive

Detailed technical and user reference

DOCUMENT CONTROL

Field	Value
Document title	Kailen AI - Complete Software Report
Source release reviewed	v2.8.0 (package.json and delivered source archive)
Platform target	Windows desktop application using Electron + Node.js
Report date	23 October 2026
Audience	End users, developer/maintainer, reviewers, testers, and future contributors
Source basis	Kailen_AI_Windows_Desktop_v2_8_Custom_Theme_Studio.zip
Verification boundary	Static source inspection, JavaScript syntax checks where available, and archive inspection; no live provider credentials or Windows installer were exercised for this report.

Important reading note This report documents the supplied source archive. Implemented source paths and declared features are distinguished from planned items and from provider or packaging behavior that has not been live-tested.

How to use this report

This is both a user manual and a technical handover. Readers who simply want to run Kailen AI should begin with the quick-start and first-launch chapters. Readers who need to understand the code should continue into architecture, data flow, local vault mechanics, API routes, and file-by-file responsibilities. Security, limitations, and validation sections intentionally state caveats plainly.

The current release is a personal, local-first desktop workspace for using AI models from multiple provider APIs. It is not a bundled subscription to those providers. API keys, model access, rate limits, and charges remain governed by the account and terms of each provider.

Contents

How to use this report . . . 2
Contents . . . 3
1. Executive summary . . . 5
2. Product concept, intended use, and scope . . . 5
3. Product evolution and release history . . . 6
4. User-facing features and navigation . . . 7
5. End-to-end user workflows . . . 8
6. Technical architecture . . . 9
7. Supported providers and model catalogue . . . 10
8. Local encrypted API-key vault . . . 11
9. Chat request lifecycle and streaming . . . 12
10. File uploads and document extraction . . . 13
11. Markdown, syntax highlighting, and safe rendering . . . 14
12. Neural Provider Hub and usage estimates . . . 14
13. Theme Studio and appearance customization . . . 15
14. Local storage and data lifecycle . . . 17
15. Local backend API reference . . . 17
16. Source tree and file-by-file guide . . . 18
17. Windows installation and first-run guide . . . 19
18. Troubleshooting guide . . . 21
19. Security, privacy, and operational considerations . . . 21
20. Implemented features, limitations, and not-yet-built capabilities . . . 23
21. Verification status and test plan . . . 23
22. Recommended roadmap . . . 24

23. Configuration and environment reference 25

24. User quick-reference checklist 25

25. Glossary 26

Appendix A. Detailed daily workflows 26

Appendix B. Responsibilities and ownership boundaries 27

Appendix C. API and storage quick reference 27

Appendix D. Maintainer handover checklist 28

Final status statement 28

1. Executive summary

Kailen AI is a Windows desktop application designed to unify access to multiple AI model providers inside a single interface. Instead of launching separate chat applications for every provider, the user can configure API credentials for supported services, choose a provider/model, send prompts, attach text-oriented documents, stream responses, and keep a local conversation history.

The project combines an Electron desktop shell, a Node.js/Express local service, and a vanilla JavaScript renderer built with HTML and CSS. Provider API credentials are stored separately from the renderer preferences in an encrypted vault file. The Provider Hub presents provider nodes in a neural-network-inspired constellation, with a detail inspector for locally recorded usage estimates and user-entered pricing assumptions.

The release reviewed here is version 2.8.0. It includes a Theme Studio in Settings for choosing a palette and adjusting custom colors, background, surface opacity, blur, corner roundness, text scale, and ambient glow. The interface also contains personalized greetings, subtle motion, chat utilities, and a local data/privacy area.

Product principle One application, several provider APIs, a consistent chat workflow, locally managed credentials, and transparent disclosure of what is only estimated.

Current product in one minute

Area	Current behavior
Desktop runtime	Electron creates a native Windows desktop window and starts the Express server on a random available loopback port.
Provider connections	OpenAI, Anthropic, Google Gemini, OpenRouter, and a custom OpenAI-compatible endpoint.
Chat	Streaming responses, Stop, regenerate, copy-answer, Markdown, code highlighting, and code-copy controls.
Documents	Up to 5 files per request; maximum 5 MB per file; supported document/code/text formats are extracted to text.
Provider Hub	Clickable provider nodes, connection state, local request count, estimated input/output tokens, models used, editable rate assumptions, and estimated spend.
Settings	Dark/light theme, six palettes, custom colors, opacity, blur, roundness, text scale, glow, compact layout, reduced motion, chat behaviors, display name, data export/clear tools.
Persistence	Encrypted vault file for provider keys; localStorage for chats, usage estimates, theme and preferences.
Known boundaries	No automatic model discovery, no authoritative billing dashboard, no model comparison, no multi-user/cloud sync, no image/audio/video understanding, and no OCR for scanned PDFs.

2. Product concept, intended use, and scope

2.1 The problem Kailen AI addresses

AI users often maintain accounts and API keys with multiple vendors. A multi-provider workflow can make it cumbersome to remember which product supports a task, keep separate histories, or switch between models. Kailen AI places provider configuration, model choice, chat, attachments, local usage estimates, and interface preferences in one desktop application.

2.2 Intended user

The current design is best suited to an individual who understands that API access is different from a consumer subscription, is comfortable obtaining API keys from providers, and wants a local desktop hub. It is not currently a multi-tenant product, team deployment system, or hosted SaaS service.

2.3 What Kailen AI is

- A multi-provider API client: a common interface for a fixed set of provider adapters.
- A local Windows application: Electron provides the desktop window while a loopback-only Express server brokers requests.
- A local-first workspace: chat history and visual preferences are stored in the Electron renderer profile; provider keys live in a separate encrypted file.
- A usage-inspection interface: a local record of prompts and responses is used to estimate token volume and cost when the user enters pricing rates.
- A customizable interface: users can choose colors, themes, and presentation preferences without editing CSS.

2.4 What it is not

- It does not provide free or included access to paid models.
- It does not log into consumer chat websites or reuse consumer subscription entitlements; its provider access is via APIs.
- It does not currently fetch official provider invoices, account balances, or exact token counts.
- It does not automatically discover every model enabled for the user account.
- It does not have production-grade user accounts, account recovery, synchronization, or collaboration.
- It is not a universal multimodal client: the current attachment extraction path focuses on text and supported office/PDF/code files.

2.5 Scope assumptions

The source is organized around a personal Windows profile. Provider API requests require an internet connection; rich Markdown formatting, syntax highlighting, and externally hosted fonts are loaded from CDNs and therefore also need internet access. The project requires Node.js 20 or newer for source/development workflows and uses Electron for its desktop shell.

3. Product evolution and release history

Kailen AI evolved from an early chat-style prototype into a provider-focused desktop workspace. This historical record summarizes the feature progression from the conversation and the release archives available for this project. Exact source validation in this report is concentrated on the v2.8.0 archive.

Release / phase	Main product change
Initial prototype	Chat-style page layout, navigation, composer, and basic interface conventions.
AI-enabled prototype	Added backend-based AI response workflow and API communication rather than a purely static mock interface.
File-upload phase	Introduced multiple file attachments and backend text extraction for supported formats.
Streaming phase	Moved from waiting for a full answer to progressive streamed responses and a Stop action.
Markdown / code phase	Added Markdown formatting, syntax-highlighted fenced code, copy-code buttons, and safer HTML sanitization.
v1.4 - feature additions	Added or consolidated conversation export, copy-answer, regenerate, theme persistence, and clear-history controls.
v1.5 - minimal UI	Emphasized bold typography, cleaner spacing, sidebar/navigation clarity, and more consistent dark/light surfaces.
v1.6 - Kailen identity / glass UI	Renamed and branded the application as Kailen AI and introduced a frosted-glass design direction.
v2.0 - multi-provider foundation	Shifted the core product from an OpenAI-centered chat clone toward one workspace for multiple provider API keys and model choices. Added local encrypted key vault concepts.

Release / phase	Main product change
v2.1 - Neural Provider Hub	Integrated a neural-network-inspired provider constellation, usage inspector, editable pricing assumptions, and provider actions.
v2.2 - Windows desktop	Added Electron shell, native desktop window, app-data vault location, and NSIS installer configuration.
v2.3 - personalized greeting	Added local-time Good morning / afternoon / evening greeting and editable display name.
v2.4 - subtle glass	Dialed back the intensity of the glass UI for improved legibility.
v2.5 - UI polish	Added compact workspace status indicators and improved hover/focus states and spacing.
v2.6 - restrained motion	Added short entrance transitions, gentle hover/press feedback, modal/toast/message motion, and reduced-motion support.
v2.7 - Settings	Added Settings page for appearance, chat preferences, personalization, and local data actions.
v2.8.0 - Theme Studio	Expanded appearance controls to six palettes, custom colors, surface opacity, blur, corner roundness, text size, and ambient background glow.

Version baseline The source archive reviewed for this report declares package version 2.8.0. Earlier version labels above summarize the iterative development history; not every historical intermediate package was re-audited as part of this report.

4. User-facing features and navigation

4.1 Main navigation

Screen	Purpose	Key controls / contents
Workspace	Primary chat area.	Time-aware greeting, summary cards, quick prompts, messages, attachment tray, model chip, send/stop buttons.
Provider Hub	Configure providers and inspect local usage.	Neural node map, connected/disconnected state, provider inspector, API-key controls, local usage estimate, price inputs, provider cards.
Usage & Status	Quick local health overview.	Vault status, connected-provider count, conversation count, and a notice that usage/billing are not provider-verified.
Settings	Customize the app and manage local data.	Theme Studio, toggles, display name, export conversations, vault lock, clear chats, reset usage.

4.2 Main workspace

The workspace is the default screen after opening the desktop application. It provides a welcome message, quick action cards (Write, Code, Learn, Explore), optional status indicators, the conversation area, and a composer docked near the bottom. After a message has been sent, the welcome panel is hidden and conversation messages occupy the central area.

The greeting uses the local system time: before 12:00 it displays “Good morning”; from 12:00 to 16:59 it displays “Good afternoon”; from 17:00 onward it displays “Good evening”. The display name defaults to “Chaitanya” in the current source and can be changed through Settings. The time is checked when the app initializes; the source does not continuously update the greeting at the minute the daypart changes while the app is already open.

4.3 Message actions

- Copy answer: copies the assistant response text through the Clipboard API when supported.
- Regenerate: removes trailing assistant messages and sends the latest user prompt again; the current implementation regenerates text prompt content rather than replaying the original file bytes.

- Stop: aborts the browser fetch request. The server observes a closed response and propagates cancellation using an AbortController.
- Copy code: every rendered fenced code block can show a small button that copies the exact code text.
- Chat list: recent conversations are kept in localStorage and can be reopened from the left sidebar.

4.4 File attachment workflow

The file picker permits multiple files. The front end checks that no more than five are attached in one message and that each selected file is at most 5 MB. Files are read in the renderer using FileReader and held in memory as Base64 values until the message is submitted. Removing a chip removes that pending attachment before submission.

4.5 Provider selection and model selection

The top provider pill can switch between connected providers and then choose a model using JavaScript prompt dialogs. Provider cards also include actions to connect/replace a key, remove a key, and open the model list. The model catalogue is fixed in server.js. This works as a functional selector, though it is less polished than a custom dropdown: model discovery is not dynamic, and a provider can offer models that differ from the hard-coded list.

5. End-to-end user workflows

5.1 First launch and vault setup

1. Launch the application using npm start or the installed Windows shortcut.
2. Kailen AI opens its local desktop window and requests the vault status from the local server.
3. If no encrypted vault exists, the user is prompted to create one with a passphrase of at least 10 characters. If a vault exists, the user must unlock it.
4. After successful setup/unlock, provider status is loaded without returning raw API keys to the renderer.
5. Open Provider Hub and connect an AI provider before submitting a chat request.

5.2 Connect a provider

1. Open Provider Hub from the sidebar.
2. Choose Connect API key for a provider node/card.
3. Paste the API key. For a custom OpenAI-compatible service, enter the base URL as well; only enter endpoints you trust.
4. Submit the form. The key is sent to the local backend, saved into the in-memory vault object, then encrypted and written to the vault file.
5. The provider card should display Connected if the local vault reports a stored key. This means a key has been saved, not that the key has been successfully tested against the provider.

5.3 Send a message

1. Ensure the vault is unlocked and the selected provider is connected.
2. Choose the provider/model through the top provider pill, or use provider actions.
3. Type a prompt. With Enter-to-send enabled, press Enter to send and Shift+Enter to insert a line break. The send button is also available.
4. Optionally attach files that fit the supported types/count/size limits.
5. The renderer serializes the chat history and attachments to the local /api/chat endpoint.
6. The backend validates the request, extracts file text where required, and starts a streaming provider request.
7. Text chunks arrive as server-sent events; the UI re-renders the progressively growing Markdown response.
8. At completion the full response is saved to local chat history, and a local usage estimate is recorded.

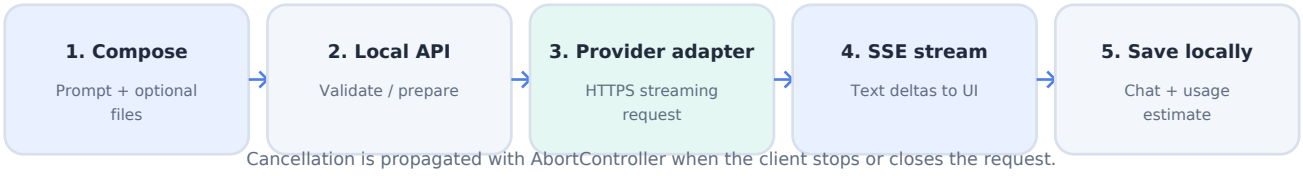


Figure 1. Typical request lifecycle from composition through provider streaming and local persistence. Provider-specific APIs are contacted only after the user submits a request.

5.4 Remove or replace a provider key

Use the Provider Hub card to replace a credential or remove it. Removal is performed by DELETE /api/providers/:provider after a confirmation. It edits the encrypted vault and should not affect charges already recorded at the provider. A provider dashboard remains the source of truth for account billing.

5.5 Personalize the application

Open Settings to change the visual palette or individual appearance values. Built-in palettes and custom values apply immediately. Preferences are stored in the local renderer profile. Reset theme returns the appearance settings to the default Midnight configuration and selects dark mode. Native operating-system chrome such as the Windows title bar remains controlled by Electron/Windows and is not fully recolored by the page theme.

6. Technical architecture

Kailen AI uses a compact three-layer architecture: the Electron shell supplies the Windows window and process lifecycle; the renderer runs the HTML/CSS/JavaScript interface; and a local Express service handles the vault, provider adapters, file extraction, and streamed chat proxy. The service is bound to loopback and is not designed as a publicly exposed API server.

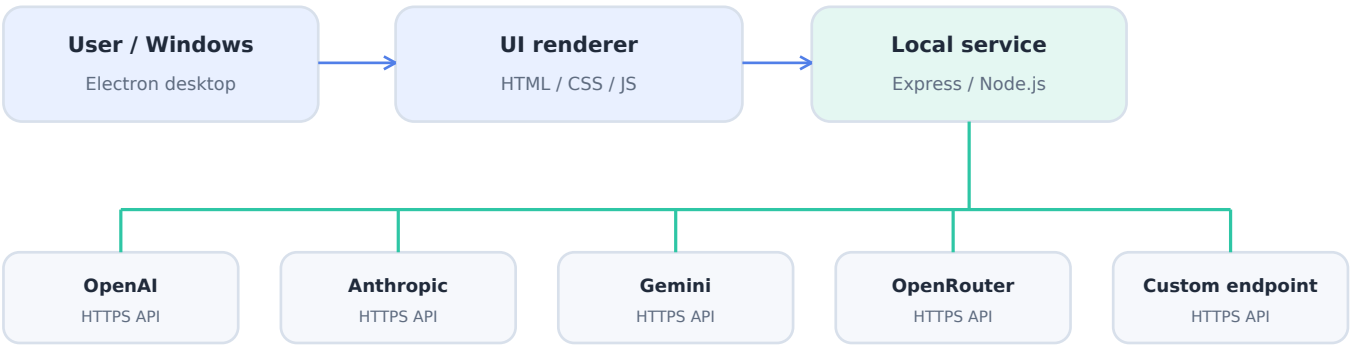


Figure 2. High-level runtime architecture and external provider connections.

6.1 Electron main process

main.js imports Electron’s app, BrowserWindow, and shell modules. It points the vault data directory at Electron’s per-user userData location, imports the Express app, and starts it on an ephemeral port bound to 127.0.0.1. Once the server is listening, it opens the browser window at that local URL. On quit, it closes the HTTP server. The default window is 1500 x 960 with minimum size 980 x 680.

The BrowserWindow enables context isolation, disables Node integration in the renderer, and enables sandboxing. New window requests are denied except for HTTPS URLs, which are opened externally through the shell. These choices reduce renderer access to Node.js APIs, but do not make all web content automatically safe; package hygiene, HTML sanitization, key handling, and trusted endpoint choices still matter.

6.2 Express service

server.js exports an Express app when required by main.js, and can also run directly in browser-development mode with npm run start:web. In desktop mode main.js selects an ephemeral port. In web mode the defaults use PORT 3000 and HOST 127.0.0.1, overridable through environment variables.

6.3 Renderer

public/index.html defines the app structure and loads public/style.css and public/app.js. The renderer uses vanilla JavaScript and DOM APIs. Markdown libraries are loaded from CDN scripts: Marked parses Markdown, Highlight.js colors code syntax, and DOMPurify sanitizes parsed HTML. Those scripts and the externally hosted font stylesheet require internet access for the full visual behavior.

6.4 Provider adapters

The backend has separate streaming functions for OpenAI-compatible Chat Completions, Anthropic Messages, and Google Gemini streaming generation. OpenRouter and custom OpenAI-compatible endpoints use the OpenAI-compatible adapter. The backend converts provider-specific stream shapes into a common delta event for the renderer.

7. Supported providers and model catalogue

Provider ID	Provider display name	Configured models in source	Protocol path
openai	OpenAI	gpt-4.1-mini; gpt-4.1; gpt-4o-mini; o4-mini	OpenAI-compatible /chat/completions streaming.
anthropic	Anthropic	claude-3-5-haiku-latest; claude-3-7-sonnet-latest; claude-sonnet-4-20250514	Anthropic Messages API streaming.
google	Google Gemini	gemini-2.5-flash; gemini-2.5-pro; gemini-2.0-flash	Gemini streamGenerateContent using SSE.
openrouter	OpenRouter	openai/gpt-4.1-mini; anthropic/claude-3.7-sonnet; google/gemini-2.5-flash	OpenRouter base URL through OpenAI-compatible adapter.
custom	OpenAI-compatible API	default (placeholder model; configure a model meaningful to the endpoint if supported)	Configured base URL + /chat/completions path.

Model availability warning

The list is hard-coded in server.js and reflects what the source currently offers as choices, not a live inventory. Model IDs can be deprecated or unavailable for a particular API key. Check each provider’s current model catalogue if a request fails.

7.1 OpenAI

The backend sends a streaming Chat Completions request to the configured OpenAI API base URL. The API key is attached as a Bearer token by the local server. The UI offers a fixed list of model IDs. The app does not automatically check entitlement, current pricing, regional access, or whether a model is enabled for the account.

7.2 Anthropic

The Anthropic adapter sends a Messages API request with streaming enabled, maps the system messages into the provider’s system field, and converts content-block text deltas to the local delta event. The model list is fixed. Some provider-specific features or request parameters may not be represented by the unified message format.

7.3 Google Gemini

The Gemini adapter maps user messages to user content and assistant messages to model content, then calls the streaming generation endpoint. A system role is passed as systemInstruction when present. The API key is used server-side in the provider request URL. It is not meant to be exposed through browser JavaScript.

7.4 OpenRouter

OpenRouter uses the base URL https://openrouter.ai/api/v1 and the OpenAI-compatible stream adapter. The configured model list includes routed model identifiers. Request metadata includes an app title/reference header. Provider-specific model access and rate limits still apply.

7.5 Custom OpenAI-compatible endpoint

The custom provider is intended for services that accept an OpenAI-compatible chat-completions request. A custom base URL is stored with the key, and the server appends /chat/completions. This is not a general arbitrary-protocol adapter; services using a different API schema need a dedicated adapter or a compatibility layer. Use only endpoints you trust because the endpoint receives prompts and the configured key.

8. Local encrypted API-key vault

The provider vault keeps raw credentials out of the renderer’s localStorage and out of the ordinary provider status payload. Keys are encrypted on disk in a file named provider-vault.enc. During a session, a successful unlock decrypts the vault into backend process memory. Locking clears the in-memory vault object and passphrase references.



Figure 3. Vault encryption lifecycle in the source: passphrase-based derivation, authenticated encryption, and encrypted file persistence.

8.1 Cryptographic implementation

Component	Implementation in source	Purpose
Key derivation	Node.js crypto.scryptSync; N=16384, r=8, p=1; output 32 bytes	Derives a 256-bit encryption key from the passphrase and a random salt.
Cipher	AES-256-GCM	Encrypts the JSON vault and authenticates it so tampered ciphertext should fail decryption.
Salt	16 random bytes	Makes derived keys unique across vault encryption events.
IV / nonce	12 random bytes	Unique per encryption operation; required for the GCM invocation.
Authentication tag	Cipher-provided GCM tag	Verified during decryption.
File write	Writes a temporary file then renames to provider-vault.enc; attempts restrictive mode 0600	Reduces risk of partial vault writes and limits file permissions where supported.

8.2 Vault creation

The backend refuses to create a vault if one already exists and requires a minimum passphrase length of 10 characters. On first setup, an empty vault is created, marked unlocked in memory, encrypted, and persisted. This length check is a floor rather than a full password-strength analyzer: users should choose a long, unique passphrase and avoid reused account passwords.

8.3 Vault unlock and lock

On unlock, the application reads the encrypted file and attempts authenticated decryption with the provided passphrase. A failure returns an authentication error without showing the stored key. On lock, the server sets the vault object to an empty structure, clears the passphrase reference, marks the vault locked, and the UI returns to the unlock prompt. The credentials are not erased from the encrypted file when locked.

8.4 Where the file lives

In Electron desktop mode, main.js points KAILEN_DATA_DIR to Electron’s per-user application-data directory and appends a data directory. This is intentionally separate from the installed application archive, which may be read-only after packaging. In web-development mode without KAILEN_DATA_DIR, server.js falls back to the project’s data directory.

8.5 Security limitations and practical rules

- Losing the vault passphrase means losing access to keys unless the user has an independent backup of both the vault and passphrase.
- A local application cannot protect secrets from malware or a sufficiently privileged user on the same machine.
- The current design has no system-keychain integration, biometric unlock, automatic idle-lock policy, or account recovery flow.
- The local service is loopback-bound, which limits network exposure, but local browser/app processes may still reach loopback endpoints. Avoid running untrusted local software while the vault is unlocked.
- Use only trusted custom API endpoints and do not paste API keys into screenshots, logs, or support messages.
- The key is held in server process memory while unlocked. Lock the vault and close the application when finished on a shared computer.

9. Chat request lifecycle and streaming

9.1 Request construction

When a user sends a prompt, the renderer verifies that the vault is unlocked and that the selected provider is connected. It creates or updates a local conversation object, appends the user’s message, stores the conversation locally, and then requests /api/chat with the selected provider ID, model ID, prior messages, and pending attachments.

9.2 Request validation and message bounds

The server requires the vault to be unlocked. It validates the provider and model field, accepts only system/user/assistant roles with string content, keeps the latest 14 messages, and truncates each message content to 16,000 characters. The final message must be a user message. The app does not implement a sophisticated conversation summarizer, tool-call protocol, or provider-independent context-window management.

9.3 Server-Sent Events

The server returns content as an HTTP response with Content-Type text/event-stream. Events are JSON payloads separated by blank lines. A delta event carries a text chunk; done indicates successful stream completion; error reports a stream error. The browser reads the response stream, incrementally decodes bytes, parses events, and re-renders the accumulated answer.

9.4 Stopping generation

The UI swaps Send for Stop while a response is in progress. Stop aborts the fetch with an AbortController. On the server, a response-close listener aborts the upstream provider request when the client disconnects before the local stream finishes. Exact cancellation behavior ultimately depends on the upstream API and transport.

9.5 Failure handling

Provider HTTP failures are converted into status-bearing errors before streaming begins, or an error event once the SSE stream has begun. The renderer shows request errors in the assistant area and a toast. Authentication problems, invalid model IDs, quota problems, rate limits, and provider outages remain possible because the service depends on remote services and credentials.

10. File uploads and document extraction

Setting	Limit / behavior
Number of files	Maximum 5 attachments per message.
Per-file bytes	Maximum 5 MiB; the frontend and backend both enforce a 5 MB limit (frontend uses 5*1024*1024 bytes).
Body size	Express JSON parser is configured for a 40 MB request body to allow base64 payloads.
Per-file extracted text	PDF/DOCX/text extraction output is capped at 30,000 characters per file.
Total attachment text	The joined attachment text is truncated to 80,000 characters before adding to the final user prompt.
Extraction location	Node.js backend memory, using pdf-parse, mammoth, or UTF-8 decoding.
Persistence	The code does not intentionally write uploaded document files to disk; the renderer holds pending file data in memory until the request is sent.

10.1 Supported extensions

Category	Extensions
Plain text / notes	txt, md, log
Structured text	csv, json, xml, yaml, yml
Code / web	js, ts, py, java, c, cpp, html, css
Documents	pdf, docx
Not supported by current extraction path	Images, screenshots, audio, video, spreadsheets such as XLSX, scanned PDF OCR, arbitrary binaries.

10.2 How the text reaches the model

The server adds extracted contents into the latest user message under a wrapper that marks the attached text as untrusted data. This is a prompt-construction convention, not a security boundary that guarantees the model ignores instructions inside files. For sensitive or adversarial documents, users should review the content and be cautious about asking a model to execute instructions found inside it.

10.3 Important limitations

- Scanned PDFs may yield little or no text because OCR is not included.
- PDF parsing quality depends on the source file, layout, and embedded text layer.
- DOCX extraction uses raw-text extraction; complex layout, tables, comments, and document styling may not be faithfully represented.

- The attachment payload and extracted text can increase API usage. Provider charges may apply to all input sent.
- The neural usage tracker estimates input tokens from text length plus attachment names; the current calculation does not include the actual extracted attachment contents, so input usage can be underestimated when files are attached.

11. Markdown, syntax highlighting, and safe rendering

Assistant messages are interpreted as Markdown. Marked converts Markdown to HTML, DOMPurify sanitizes the resulting markup, and the renderer inserts the sanitized content into the assistant message area. Highlight.js colors supported fenced code blocks. A code header shows a language label inferred from the code block's language class and a Copy code button.

11.1 Supported presentation

- Markdown headings, paragraphs, emphasis, lists, links, blockquotes, and GitHub-flavored tables as parsed by Marked.
- Inline code and fenced code blocks with horizontal scrolling.
- Syntax colors supplied by a Highlight.js stylesheet hosted through cdnjs.
- Copy answer and Copy code actions, with feedback such as "Copied!" where clipboard access succeeds.
- Fallback output escapes text if Markdown or sanitizer libraries did not load.

11.2 Security behavior

DOMPurify reduces risks from inserting assistant-generated HTML, and user messages are added with textContent rather than interpreted as HTML. The renderer's security also depends on keeping the CDN assets trustworthy and updated, not introducing unsafe string-based UI changes, and maintaining Electron's context isolation and disabled Node integration. This is not a substitute for a formal security audit.

11.3 Dependency/network implication

The frontend loads Marked, Highlight.js, DOMPurify, the Highlight.js theme, and web fonts from external CDNs. Without internet access these features may fail to load, though the app can fall back to escaped text in the Markdown path. An offline distribution should vendor pinned versions locally and review license/upgrade policy.

12. Neural Provider Hub and usage estimates

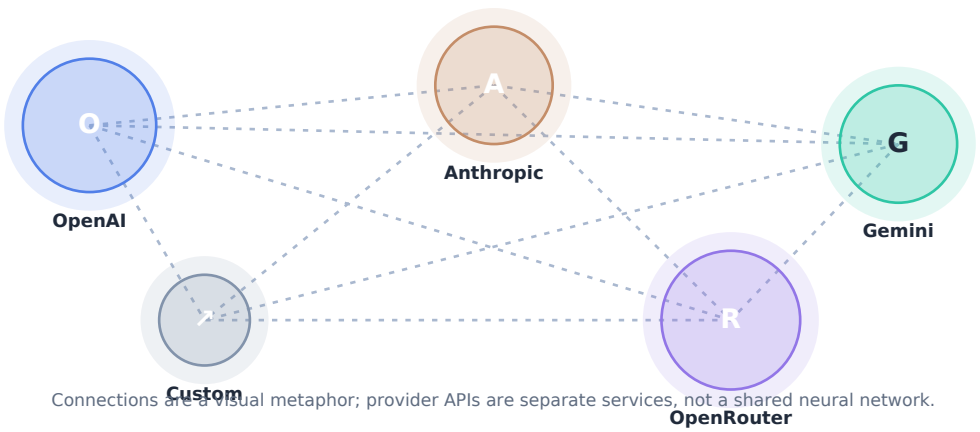


Figure 4. Conceptual view of the provider constellation. The connecting lines are a visual motif, not a claim that providers form one shared runtime network.

12.1 Node visualization

The Provider Hub renders one SVG node for each configured provider. Node radius is based on locally estimated input plus output token totals, scaled against the provider with the largest recorded total. Providers without recorded use still have a base node size. Lines connect the nodes as a visual representation of the user's unified workspace; the underlying provider services do not call each other through those lines.

12.2 Selected-node inspector

Clicking or keyboard-selecting a node opens its inspector. It displays connection status, the local request count, approximate input tokens, approximate output tokens, estimated spend, model activity call counts, editable input/output pricing assumptions, provider billing guidance, and actions for connecting/replacing a key, using the provider, or resetting its local usage data.

12.3 Usage estimate calculation

```
estimatedInputTokens = ceil(estimatedInputText.length / 4)
estimatedOutputTokens = ceil(assistantResponse.length / 4)
estimatedCostUSD = (inputTokens * inputRate + outputTokens * outputRate) / 1,000,000
```

Input and output rates are user-entered values in USD per one million tokens. This heuristic is an approximation based on string length; actual tokenization varies by language, model, encoding, and provider. It is especially likely to undercount file-based input because attachment extraction happens on the backend after the renderer's current local usage estimate has assembled the prompt text plus filenames.

12.4 What the chart does not do

- It does not call official provider usage or billing APIs.
- It does not show remaining account balance, invoices, credits, or exact token usage.
- It does not account for every provider-side pricing rule, cached input discount, tool charges, image/audio tokens, or model-specific pricing differences.
- It does not dynamically discover models. The model activity report only records locally used model IDs and call counts.
- Resetting local usage deletes the app's local estimate record only; provider accounting is unchanged.

Treat as an estimate	Use the Provider Hub to compare your own local usage trends, not as an invoice or financial ledger. Confirm real charges in each provider's own dashboard.

13. Theme Studio and appearance customization

The Settings section includes an Appearance area that applies visual changes immediately. These choices are saved to localStorage in the current Electron renderer profile. Most page surfaces use CSS custom properties, so the interface can be changed without replacing the stylesheet or restarting the app.

13.1 Built-in palettes

Palette	Visual character	Typical use
Midnight	Navy/blue with violet secondary accent	Default balanced dark workspace; light variant remains cool and clean.
Graphite	Neutral charcoal and steel	Lower-color, technical aesthetic.
Aurora	Deep teal with aqua and blue	Teal highlights and a calm modern tone.
Forest	Forest green with soft yellow-green accents	Nature-inspired palette.
Amethyst	Deep purple, violet, and soft magenta	Purple-centric appearance.
Sand	Warm brown, amber, and terracotta	Warm, earth-toned workspace.

13.2 Customization controls

Control	Current behavior / range
Color scheme	Dark or light. The top-bar toggle switches the selected palette’s light/dark variant.
Accent color	Preset default / blue / violet / mint / rose. A custom color picker can override the accent.
Background color	Color input sets the workspace base background. Leaving it unset uses the preset background.
Surface opacity	45% to 100%. Lower values appear more translucent; higher values look more solid.
Glass blur	0 to 24 pixels. Lower values reduce background blur; zero effectively removes blur.
Corner roundness	8 to 26 pixels; affects key cards, panels, and controls using shared radius variables.
Text size	90% to 115% scale, applied to body text and selected component text groups. Not every single label has its own scale rule.
Background glow	Toggle the soft ambient gradient behind the workspace.
Reset theme	Restores Midnight palette defaults, clears custom colors, resets sliders, enables glow, and switches to dark mode.

13.3 Other Settings

- Compact layout: reduces spacing in selected sidebar, list, quick-card, and settings areas.
- Reduce motion: attaches a dataset flag that disables animations/transitions in CSS.
- Enter to send: when enabled, Enter sends and Shift+Enter inserts a newline; when disabled, Enter inserts a newline and the send button remains available.
- Show workspace overview: hides/shows the home status strip and workspace footnote.
- Confirm before clearing chats: lets the user skip confirmation prompt for clearing history if disabled.
- Display name: stores a local name and updates the greeting.
- Export conversations: downloads the locally saved conversation list as a JSON file.
- Lock vault: invokes the normal lock workflow; if already locked, it opens the unlock dialog.
- Clear chats: empties local conversation history.
- Reset usage estimates: removes the local neural usage tracking record only.

14. Local storage and data lifecycle

Data	Location / key	Contents	Security / behavior
Provider keys	Encrypted file provider-vault.enc under Electron userData/data	Provider ID, API key, optional base URL	AES-256-GCM encrypted. Decrypted into server memory when unlocked.
Chat history	Renderer localStorage key kailen-chats	Recent conversation IDs/titles and role/content message records	Not encrypted by the app. Protected only by current OS account/profile controls and device security.
Theme preference	Renderer localStorage key kailen-theme	dark/light selection	Not secret.
Settings	Renderer localStorage key kailen-settings-v1	Palette, color overrides, blur, opacity, radius, text scale, toggles	Not secret; current Electron profile scope.
Display name	Renderer localStorage key kailen-user-name	Greeting name	Not secret.
Neural usage	Renderer localStorage key kailen-neural-usage-v1	Local request/token estimates, per-model counts, entered price rates	Not an official billing ledger; reset from the Hub or Settings.
Pending attachments	Renderer memory until submit; Base64 in request body	Attachment bytes and names	Not intentionally persisted to local disk by the app; sent to selected provider when submitted.

14.1 Chat history capacity

The app saves the most recent 40 conversations to localStorage. The archive export contains the current locally loaded conversation array and explicitly excludes API keys. Because browser/Electron localStorage has finite capacity, large histories may encounter storage quota errors; the UI asks the user to export important chats if storage fills.

14.2 Separation of secrets and preferences

Theme controls, display name, and local chat history are not stored inside the provider vault. Conversely, provider keys are not returned by the status endpoint. This separation is useful but does not mean all local data is encrypted: only the provider vault is encrypted by this implementation. Chat content may include personal data and is visible to anyone with access to the same unlocked operating-system account/profile.

15. Local backend API reference

The following endpoints are implemented in server.js. In desktop mode the service listens on an ephemeral loopback port chosen by Electron, so scripts should not assume port 3000. In browser-development mode, the default port is 3000. These endpoints are intended for the app’s local renderer, not for Internet exposure.

Method + path	Authentication/state	Purpose / behavior
GET /api/health	No vault unlock required	Returns liveness, app name, vault presence/unlocked state, and provider status if unlocked. Does not return raw API keys.
GET /api/vault/status	No vault unlock required	Returns whether encrypted vault exists, whether it is unlocked, and whether environment passphrase config is present.
POST /api/vault/setup	First-time setup	Accepts passphrase; requires at least 10 characters and no existing vault; creates encrypted empty vault.
POST /api/vault/unlock	Existing vault	Accepts passphrase; decrypts provider vault or reports failure.
POST /api/vault/lock	No body required	Clears in-memory credentials/passphrase and marks vault locked.

Method + path	Authentication/state	Purpose / behavior
GET /api/providers	Vault must be unlocked	Returns safe provider status (connected flag, base URL) without the API key.
POST /api/providers/:provider	Vault must be unlocked	Adds/replaces API key and optional base URL for supported provider ID; encrypts vault.
DELETE /api/providers/:provider	Vault must be unlocked	Removes a provider key and re-saves the encrypted vault.
GET /api/models	No vault unlock required	Returns the hard-coded model catalogue for each provider.
POST /api/chat	Vault must be unlocked	Validates messages, extracts optional attachments, streams a provider answer via SSE.

15.1 Chat request shape (conceptual)

```
POST /api/chat
Content-Type: application/json

{
  "provider": "openai",
  "model": "gpt-4.1-mini",
  "messages": [
    {"role": "user", "content": "Explain recursion simply."}
  ],
  "attachments": []
}
```

15.2 Stream event envelope

```
event: delta
data: {"text":"A recursive function..."}

event: done
data: {"ok":true}
```

Errors before the SSE response is opened are returned as JSON with an HTTP status. After streaming begins, failures are emitted as an error event. The UI uses the stream event name plus its JSON data to build the response.

16. Source tree and file-by-file guide

```
kailen_ai_v2/
  main.js                 Electron desktop entry point
  server.js               Express API, vault, extraction, provider adapters
  package.json            Dependencies, scripts, Windows builder config
  .env.example            Optional local environment configuration
  .gitignore              Excludes secrets, data files, node_modules
  README.md               Install, usage, limitations, release notes
  data/
    .gitkeep              Keeps data directory in source archive
  public/
    index.html            Application screens and modals
    style.css             Design tokens, layout, responsive UI, themes
    app.js                UI state, settings, chat, provider hub logic
    old-app.js             Previous/legacy frontend file, not the active app.js entry point
    old-style.css          Previous/legacy stylesheet, not the active stylesheet link
```

16.1 main.js

Creates the Electron BrowserWindow, sets its initial and minimum size, enables context isolation, disables renderer Node integration, turns on sandboxing, opens HTTPS popups externally, starts the Express service on 127.0.0.1 with a free port, and closes the service during app shutdown. It also configures the encrypted vault path in the per-user application-data folder.

16.2 server.js

This is the security- and provider-sensitive service layer. It owns environment config, Express routes, scrypt/AES-GCM encryption, safe provider status, a static model list, file-size/type checks, PDF/DOCX/text extraction, SSE event generation, and the streaming adapters for OpenAI-compatible endpoints, Anthropic, and Gemini.

16.3 public/index.html

Defines the sidebar, navigation views, greeting, workspace status strip, message composer, attachment input, Provider Hub network SVG, provider cards, usage page, Settings view, vault setup/unlock modal, API key modal, and toast. Scripts are loaded with defer so the DOM is parsed before app.js executes.

16.4 public/style.css

Defines theme variables, typography, glass surfaces, card layouts, message styles, responsive adjustments, animation states, reduced-motion behavior, Settings controls, palette swatches, and CSS variables updated by the Theme Studio. Theme variables include background, panel colors, text, border, accent, blur, corner radius, and font scaling.

16.5 public/app.js

Manages the renderer state and user interactions: greeting, local preferences, settings event handlers, theme palette application, chat persistence, vault modal, provider CRUD, provider/model selection, Markdown sanitization, code decoration/copy buttons, attachments, streaming, regeneration, chat export, and local neural usage estimates.

16.6 package.json and .env.example

package.json declares version 2.8.0, Electron start scripts, Express web-mode script, the electron-builder NSIS Windows installer configuration, runtime dependencies, and Node.js engine floor. .env.example contains local host/port settings and comments about optionally configuring a vault passphrase. It contains no real API key.

17. Windows installation and first-run guide

17.1 Requirements

- Windows PC supported by Electron’s x64 build path.
- Node.js 20 or newer for source installation/build.
- Internet access to install packages, download Electron/build dependencies, load CDN-hosted Markdown/highlight/font assets, and call provider APIs.
- An API account/key for any provider the user wants to use. API usage may be paid.
- Sufficient disk space for node_modules and Electron packaging outputs.

17.2 Run from source

1. Download the current source ZIP and extract it to a folder you control.
2. Open the extracted kailen_ai_v2 folder in VS Code or PowerShell.
3. Run npm install and allow dependency installation to finish.
4. Run npm start. The Electron window should open once the local server is ready.
5. On the first launch, create a vault passphrase of at least 10 characters. Record it securely; it is not recoverable by the app.
6. Open Provider Hub and connect an API key. Use a valid provider key and a model that is enabled for your account.
7. Return to Workspace, choose the provider/model, enter a prompt, and submit it.
8. Use Settings to personalize the interface and export conversations periodically.

17.3 Browser development mode

Run npm run start:web from the project folder. The Express service binds to 127.0.0.1 by default and listens on port 3000 unless PORT is set. Open http://127.0.0.1:3000. The desktop workflow is preferred for everyday usage because it stores the vault under Electron’s per-user application-data directory.

```
# From PowerShell, inside kailen_ai_v2
npm install
npm start

# Optional browser-only development mode
npm run start:web
```

17.4 Build a Windows installer

```
npm install
npm run dist:win
```

The electron-builder configuration targets a 64-bit Windows NSIS installer. The output folder is dist, with installer naming based on product name/version/builder behavior. Building the installer requires package downloads and should be run on Windows for the intended Windows distribution. The source ZIP is not itself a compiled installer.

17.5 Backups and upgrades

Before replacing versions or uninstalling, close the app and back up the encrypted vault file from Electron’s userData/data folder. Also consider exporting conversations from Settings. Preserve the vault passphrase separately. Theme settings and chat data are kept in the renderer profile; they may persist across an app upgrade that uses the same Electron app identity/profile, but exporting is safer than relying on implicit profile preservation.

18. Troubleshooting guide

Symptom	Likely causes	Recommended checks
App does not start	Node/Electron install incomplete, dependency error, or unsupported Node version.	Check Node version (20+), run npm install again, start from terminal, capture the full error.
Vault asks to unlock every time	Existing vault remains locked until passphrase is entered; wrong profile/data directory could look like a different installation.	Use the same Windows user/profile and passphrase. Do not delete provider-vault.enc.
Lost vault passphrase	The vault cannot be decrypted without the original passphrase.	Look for a secure backup of both encrypted vault and passphrase; the software has no recovery mechanism.
Provider shows Connected but chat fails	Connection state only indicates a key was saved; it does not validate the key/model.	Check provider key, API permissions, model identifier, quota/billing, region and latest provider API rules.
401 / 403 response	Invalid/expired key or insufficient access.	Create/rotate a key in the provider portal, then replace it in Provider Hub. Never paste the key into a public ticket.
404 / model not found	Static model list may be outdated or the model is not available to the key.	Check the provider's live model catalog and account access. Update source model list if needed.
429 or quota error	Rate limits or exhausted provider quota.	Check provider dashboard, billing, retry timing, and limits. Kailen AI does not override provider quotas.
Custom provider fails	Base URL or API schema is not OpenAI-compatible, or the final /chat/completions path is wrong.	Enter the correct base URL and use a compatible endpoint. A non-compatible API needs a new adapter.
No syntax colors / Markdown styling	CDN blocked/offline, DNS issue, or browser cannot load assets.	Check internet access and developer console. Consider vendoring pinned library files for offline use.
PDF upload has empty text	Scanned PDF, image-only pages, or complex PDF text layout.	Use a text-searchable PDF or add OCR in a future version.
File upload rejected	Unsupported extension, more than five attachments, file over 5 MB, or malformed payload.	Use supported file types and stay within per-file/count limits.
Copy button does not work	Clipboard access may require secure context/permission or app policy.	Try selecting text manually; test under the supported Electron/localhost context.
Usage estimate looks too low/high	Character-count-based token estimation and user-entered rates are approximate.	Treat it as a trend indicator only; confirm costs using provider billing records.
Theme custom color not fully reflected	Native title bar is controlled by the OS; CSS variables cover web content only.	Adjust page controls and remember that OS chrome is outside page theming.

19. Security, privacy, and operational considerations

19.1 Positive security design choices present in source

- Provider keys are kept in a separate encrypted vault file rather than in localStorage.
- AES-256-GCM provides authenticated encryption; scrypt derives the encryption key from the passphrase.
- The app uses a loopback-only local service; desktop mode picks an ephemeral port on 127.0.0.1.
- Electron context isolation is enabled, Node integration is disabled, and sandboxing is enabled.
- New windows are denied by default; HTTPS links can open externally.
- DOMPurify sanitizes assistant Markdown output before it is inserted into the message DOM.

- The Provider Hub status response omits raw API key values.

19.2 Important risk areas

- Chat history is stored in localStorage without application-level encryption.
- Prompts, conversation history sent for context, and extracted attachment content are transmitted to the selected provider when submitted.
- A custom endpoint receives the request body and configured API key; verify endpoint ownership and TLS before using it.
- Provider data handling, retention, and training policies vary by service and account settings; consult each provider’s applicable terms and privacy controls.
- The backend has no full user-authentication or session system; it is intended for one local user and local-only use.
- The vault has no forgotten-passphrase recovery feature or OS keychain integration.
- No formal penetration test, code audit, or independent cryptographic audit is implied by this report.
- CDN-hosted front-end libraries introduce external availability and supply-chain considerations.

19.3 Recommended operating hygiene

1. Keep Windows and Node/Electron dependencies updated, and review changes before upgrading.
2. Never change HOST to 0.0.0.0 or expose the local server to the public network as-is.
3. Keep provider keys in the vault, not in source code, screenshots, exported chat files, or committed .env files.
4. Use a unique, long vault passphrase and store it in a reputable password manager.
5. Lock the vault and close the application when using a shared or unattended PC.
6. Use least-privilege API keys and provider spending limits/alerts where the provider offers them.
7. Use the Settings export as a convenience backup but remember it contains conversation content and should be protected accordingly.
8. Verify real usage and bills on provider dashboards rather than relying on local estimates.

Data-sharing reminder	When a prompt is submitted, the selected provider receives the conversation context and any supported extracted file text included in that request. “Local-first” refers to the application and its storage design; it does not mean prompts stay on the computer after provider submission.

20. Implemented features, limitations, and not-yet-built capabilities

Capability	Current status	Notes
Windows desktop shell	Implemented in source	Electron window and NSIS build configuration exist; packaged installer not verified during report preparation.
Provider key vault	Implemented in source	scrypt + AES-256-GCM; no keychain or recovery feature.
Provider integrations	Implemented in source	OpenAI, Anthropic, Google Gemini, OpenRouter, custom OpenAI-compatible. Live provider calls not verified for this report.
Provider/model selector	Implemented with limitations	Current top provider pill uses prompt dialogs; catalogue is static.
Streaming responses	Implemented in source	SSE delta events and Stop via AbortController. Upstream cancellation depends on provider.
Files and extraction	Implemented in source	Text-oriented uploads, PDF/DOCX/text extraction, set limits. No OCR or image/audio/video.
Markdown and code	Implemented in source	CDN-loaded Marked, DOMPurify, Highlight.js. Offline rich formatting needs local assets.
Neural Provider Hub	Implemented in source	Interactive SVG nodes and inspector, local counts/rates/cost estimates. Connections are visual.
Customizable theme	Implemented in source	Six presets plus custom background/accent and sliders. Native Windows chrome excluded.
Usage/cost authority	Not implemented	No official billing APIs, account balance, invoice, exact token accounting.
Model auto-discovery	Not implemented	Hard-coded IDs; no live capabilities/catalogue fetch.
Side-by-side response comparison	Not implemented	Explicitly excluded from present scope.
Accounts / sync / collaboration	Not implemented	No login service, cloud sync, or shared workspace.
Multimodal input	Not implemented	No image, audio, or video input path.
OCR	Not implemented	Scanned/image-only PDFs are unsupported by the current extraction path.
Production hardening	Needs future work	Rate limits, authentication, formal security testing, and installer validation should be considered before broader distribution.

21. Verification status and test plan

This documentation build inspected the v2.8.0 source archive, read the package metadata and code paths, and documented the application based on the delivered source. Earlier project delivery steps reported JavaScript syntax checks and ZIP integrity checks for the relevant release. This report itself does not claim that every feature was exercised in a live Electron window, that the installer was built, or that the remote provider APIs were called with the user’s accounts.

21.1 Static validation checklist

Check	What it can establish	What it cannot establish
node --check main.js	JavaScript syntax is parsable.	Electron launch success, runtime DOM behavior, or OS integration.
node --check server.js	Backend syntax is parsable.	Provider key validity, network reachability, billing, or API schema compatibility.
node --check public/app.js	Renderer JavaScript syntax is parsable.	Settings visuals, all interactions, CDN loading, or accessibility quality.
ZIP archive test	ZIP directory and compressed file data are structurally readable.	The app starts correctly after extraction or packaging.
Live smoke test (recommended)	Real startup, vault creation/unlock, connection, chat stream, uploads, theme controls, persistence and installer behavior.	Comprehensive security assurance or performance across all systems.

21.2 Recommended acceptance test

1. Install Node.js 20+ and run npm install from a clean extraction.
2. Run npm start; verify the Electron window opens and the vault setup prompt appears.
3. Create a vault with a unique passphrase; close/relaunch; confirm unlock works and a wrong passphrase fails.
4. Connect one provider key; verify the provider appears connected without exposing the key in status responses.
5. Send a short prompt and verify streamed text, Stop, Copy answer, Regenerate, and saved conversation replay.
6. Attach a text file and a text-searchable PDF; verify the response refers to their contents; confirm oversized/unsupported files are rejected.
7. Open Provider Hub; click each node; edit rates and confirm local estimates display; reset local usage and verify it clears.
8. Open Settings; test each palette, custom colors, background, sliders, glow toggle, compact layout, reduced motion, Enter-to-send, display name, export, clear, and vault lock.
9. Run npm run dist:win on Windows; install and launch the generated installer output; verify per-user vault and preferences survive a restart.
10. Check the provider’s official dashboard separately to reconcile actual charges.

22. Recommended roadmap

The following roadmap is a recommendation, not a statement of implemented behavior. Priorities are ordered to improve reliability, visibility, and safe day-to-day usage before adding large new feature categories.

Priority	Enhancement	Why it matters / suggested approach
P0	Real provider/model verification	Add a “Test connection” action that sends a low-cost verification request or model-list request only when supported; clearly distinguish “key saved” from “connection tested”.
P0	Provider/model selector redesign	Replace window.prompt dialogs with a native in-app popover/dropdown listing connected providers, model ID, and any known context/pricing metadata.
P0	Security hardening pass	Audit all local endpoints, request sizes, custom endpoint configuration, error handling, dependency versions, and renderer sinks. Consider a per-launch random secret/origin guard for local API requests.
P1	More precise token tracking	Use provider-reported usage metadata where returned by the API; clearly label estimated vs provider-reported values and handle cached/output token categories.

Priority	Enhancement	Why it matters / suggested approach
P1	Billing integrations	Optional read-only provider billing/usage connectors where APIs and permissions exist; never imply every provider offers the same billing fields.
P1	Automatic model discovery	Fetch model catalogues when supported, cache results, and retain manual model entry for custom endpoints.
P1	Offline assets	Bundle pinned Marked, DOMPurify, Highlight.js, and fonts locally for more reliable offline rendering and lower CDN supply-chain exposure.
P1	OS credential protection	Offer Windows Credential Manager/DPAPI-backed key wrapping or another OS-backed key store as an option, with a documented threat model.
P2	SQLite local persistence	Move conversation metadata/history from localStorage to SQLite for better scale, query, migration, export, and recovery behavior.
P2	Search and organization	Conversation search, tags, pinned chats, rename/delete individual chats, and archive folders.
P2	Prompt and context management	Editable system prompts, per-provider generation settings, context-window warnings, usage limits, and conversation summarization.
P3	Multimodal support	Add image and audio support with provider capability checks; implement OCR as a separate explicit optional path.
P3	Packaging automation	Build/test the installer in CI on Windows, generate checksums, retain release notes, and test upgrades from previous versions.

23. Configuration and environment reference

Variable	Default in source	Meaning / guidance
PORT	3000	Web-development mode port. Desktop mode starts Express on an ephemeral free port.
HOST	127.0.0.1	Loopback-only host default. Do not expose the local server publicly.
KAILEN_DATA_DIR	Electron userData/data in desktop mode; ./data in direct web mode	Directory for provider-vault.enc. Back up this directory with a secure copy procedure.
KAILEN_VAULT_PASSPHRASE	Unset unless configured	Optional server environment passphrase for vault setup/unlock path. For personal use, prefer the UI-created passphrase and never commit a real value into source control.
OPENAI_API_KEY	Not used as a provider configuration requirement in current UI flow	Current UI manages provider keys in the encrypted vault rather than relying on a single API key environment variable.

The supplied .env.example comments focus on PORT, HOST, and an optional vault passphrase. API keys for provider connections are submitted through Provider Hub and saved in the encrypted vault.

24. User quick-reference checklist

Everyday use

- Launch Kailen AI and unlock the local vault.
- Check the provider/model shown in the top bar.
- Submit prompts using Enter or the Send button; use Shift+Enter for newlines when Enter-to-send is enabled.
- Attach no more than five supported files, each no larger than 5 MB.
- Use Stop if the response should be cancelled.

- Use Copy answer or Copy code when needed.
- Use Provider Hub for key management and local usage estimates.
- Use Settings for appearance, chat behavior, greeting, export, vault lock, and local cleanup.
- Verify actual cost with the provider billing dashboard.

Before upgrading / uninstalling

- Close Kailen AI.
- Securely back up provider-vault.enc and ensure the passphrase is preserved separately.
- Export conversations from Settings if the conversations matter.
- Remember that exports contain prompt and response text and should be protected like personal data.
- Install a new version in a separate folder first if preserving the existing project source is important.

25. Glossary

Term	Meaning in Kailen AI
API key	Secret credential issued by a provider to authenticate programmatic API requests.
Provider	External AI service such as OpenAI, Anthropic, Google Gemini, or OpenRouter.
Model	A model ID selected within a provider’s API. Availability depends on provider and account.
Adapter	Code that converts the app’s common message/stream handling into a provider-specific API format.
SSE	Server-Sent Events; a simple event-streaming protocol used to transport text deltas from the local server to the UI.
Electron	Framework that wraps web technologies in a desktop application and provides Chromium/Node process management.
Express	Node.js web framework used for the local API routes.
AES-256-GCM	Authenticated encryption mode used to encrypt the provider vault.
scrypt	Password-based key derivation function used to derive an encryption key from the vault passphrase.
localStorage	Browser/Electron renderer key-value storage used for chats, preferences, theme choice, name, and usage estimate records.
Markdown	Lightweight text markup used to format assistant responses.
DOMPurify	Library used to sanitize parsed Markdown HTML before displaying it.
Highlight.js	Library that provides code syntax highlighting.
Electron userData	Per-user directory provided by Electron for application state; used as the desktop vault location parent.
Estimate vs invoice	A local approximation is not a statement from a provider billing system.

Appendix A. Detailed daily workflows

A.1 Starting a new chat

Open the Workspace view from the sidebar. Confirm that the vault is unlocked; if not, the app will request the passphrase before a chat can be sent. Confirm that the top provider pill reflects the intended provider and model. If no key is connected, the app redirects to Provider Hub and shows a connect-provider notification. Type a prompt, attach optional files, and send. Keep Shift+Enter in mind if multi-line prompts are needed.

A.2 Working with code

Ask the selected model to generate code or explain a snippet. Fenced code blocks are converted from Markdown, sanitized, highlighted where a language grammar is available, and decorated with a Copy code button. Use Copy answer for the complete assistant text. Syntax highlighting may be absent if its CDN script could not load, although the code itself should remain readable.

A.3 Reviewing usage

Navigate to Provider Hub and select a provider node. Check the local request count and character-derived input/output token totals. Enter input/output prices per one million tokens to calculate a rough cost estimate. If more than one model with different rates has been used, the same rate fields are applied to the provider's aggregated estimate, so the estimate can be materially inaccurate. Reset local estimates when desired; doing so does not change provider records.

A.4 Customizing appearance

Use a preset first. Then select a custom accent/background color or adjust surface opacity, blur, roundness and text scale. The options update the rendered page in real time and persist in the local profile. The background-glow setting removes the ambient gradients behind the workspace. If a custom color creates poor contrast, revert to a built-in palette or use Reset theme.

Appendix B. Responsibilities and ownership boundaries

Responsibility	Owned by Kailen AI	Owned by external provider / Windows
Desktop layout, settings and navigation	Yes	Native window frame / some OS accessibility settings remain OS-managed.
Encrypting stored API keys	Yes, in local vault file	Provider issues, rotates, and revokes API keys.
Prompt preparation and stream relay	Yes	Inference, model response quality, tokenization, rate limiting and quotas.
File extraction before send	Yes, locally	Provider receives included extracted text after submit.
Usage approximation	Yes, local heuristic and user-entered prices	Official token reporting, invoices, balance, credits, tax, pricing, and account billing.
Model availability	Static list and request selection	Provider controls model availability, deprecations, and account entitlement.
System updates	Source can be updated by project maintainer	Windows/Node/Electron/CDN/provider API updates can affect compatibility.

Appendix C. API and storage quick reference

C.1 Provider identifiers

```
openai       -> OpenAI Chat Completions streaming
anthropic    -> Anthropic Messages streaming
google       -> Gemini streamGenerateContent
openrouter   -> OpenRouter OpenAI-compatible streaming
custom       -> Custom OpenAI-compatible /chat/completions
```

C.2 Local persistence identifiers

```
kailen-chats                -> recent conversations (renderer localStorage)
kailen-theme                -> dark/light scheme
kailen-settings-v1          -> theme and UI preferences
kailen-user-name            -> greeting display name
kailen-neural-usage-v1      -> local usage estimates and entered rates
provider-vault.enc          -> encrypted provider keys (server-side file)
```

C.3 Upload extension whitelist

```
txt, md, csv, json, pdf, docx, js, ts, py, java, c, cpp,
html, css, xml, yaml, yml, log
```

Appendix D. Maintainer handover checklist

- When updating supported models, change the model catalogue in server.js and verify model IDs from current provider documentation.
- When adding a provider, implement a provider metadata entry in the renderer, an allowed provider ID in the backend vault route, a model list, a streaming adapter, request parsing, and a test for error mapping.
- When changing vault crypto or data shape, add versioned migration behavior and test both valid and invalid ciphertext.
- When adding theme variables, provide sensible default and dark/light values and make sure Reset theme resets the new variable.
- When changing file uploads, keep front-end and back-end validation in sync; never trust only browser-side size/type checks.
- When changing renderer HTML, continue sanitizing generated Markdown and avoid treating user-controlled strings as HTML.
- When packaging, test the installer on a clean Windows profile and verify persistence paths after uninstall/reinstall.
- Update README and this report whenever actual source behavior changes; do not describe unimplemented features as live integrations.

Final status statement

Kailen AI v2.8.0 is a functional source-level foundation for a local Windows multi-provider AI workspace. It already includes multiple provider streaming adapters, local encrypted API-key storage, a neural-themed Provider Hub, local usage estimation, file extraction, Markdown/code rendering, a personalized greeting, animation/reduced-motion styling, settings, and a Theme Studio. Its most important boundaries are the static model catalogue, approximate local token/cost estimates, limited attachment modalities, lack of account synchronization/multi-user features, and absence of live provider-billing integration. A clean Windows runtime/install test and authenticated live provider request should be treated as explicit release acceptance steps.

End of report This document reflects the supplied v2.8.0 source archive as reviewed on 23 October 2026. The report is not a security audit, billing statement, or guarantee that every external provider endpoint and model identifier remains available.